Lab Manual : Pointer ,Dynamic 1D & 2DArray

Important Note:

- Have an improved understanding of pointers.

✓ Declaring and Initializing pointers

✓ Pointer Operations

There shouldn't be any memory leakage or dangling pointers in your program. - Make separate functions for input and output of arrays. Your main should be a sequence of function calls only

- You are not allowed to use global variables and goto instruction
- **Submit only one cpp file having main function testing all the following functions**

Task 1

Exercise 1 [Input Array]:

Write a function **int* InputArray(int& size)** that asks user to enter size of required array, allocates the memory on heap, takes input in array and returns its pointer.

Exercise 2 [Output Array]:

Write a program **void OutputArray(const int* myArray, const int& size)** that takes a pointer to an integer array and prints its data.

Write main function to test above functionality.

Exercise 3 [Compress Array]:

Write a function **int* CompressArray(int* oiginalArr, int& size)** that takes a sorted array and removes duplicate elements from this array.

Sample Run:

```
//Input:
Enter Size of array: 10
Enter 10 elements: 1 2 2 2 3 3 3 3 3 7
//Output
Array after Compression: 1 2 3 7
```

Your function will compress the original array, allocate new array of compressed size (compressed size is 4 in above example) on heap, copy updated array in new array and return the new array.

Take input from user by calling **int* InputArray(int& size)** (function you implemented in Exercise 1). Call CompressArray, call OutputArray(function you implemented in Exercise 2) to display the final output.

Note: Make appropriate functions for following problems yourself

Exercise 4 [Intersection]

Implement a function **int* Intersection(int* setA, int& size1, int* setB, int& size2, int& size3)** that finds intersection (common elements) of two sets (stored using arrays).

Sample Run:

```
//Input:
Enter Size of Array: 6
```

```
Enter 6 elements: 1 2 3 4 5 6

Enter Size of Array: 4
Enter 3 elements: 1 3 5 7

//Output
A = {1,2,3,4,5,6}
B = {1,3,5,7}
A Intersection B = {1, 3, 5}
```

Help: Note array3 should not have any duplicate elements. You have to:

· Allocate the three arrays dynamically after inputting the size of array1 and array2 from the user. Statically allocated arrays are NOT allowed

· Initially you can allocate elements = (size of array1 + size of array2) to array3. For example you would allocate 6+4 to array3 for the above example. After finding the common elements, the allocated size of array3 may be more than what you need. (In the above example you require size 3 whereas you have allocated 10).

Task 2

i) Write a function void populate_array(int **my_array,int m,int n) that initialize an array with random data.

ii) Write a function void disp_array(int **my_array,int m, int n) display an element of an array correctly (m*n format)

iii) write a function int **transpose_array(int **my_array,int m,int n) that creates a dynamic 2d array of size n*m that is the transpose of an array and returns a 2d pointer to the transpose array.

Sample input:

Enter the size of matrix m*n : 3 3

Original Matrix:

1 2 3

4 5 6

7 8 9

Transpose Matrix:

1 4 7

2 5 8

3 6 9

Task 3

i) Write a function char** AllocateMemory(int& rows, int& cols) that takes the size of matrix (rows and columns) from the user, allocates memory for the matrix and returns its pointer.

ii) Write a function void InputMatrix(char** matrix, const int rows, const int cols) which takes input the values in matrix from user(console).

iii) Write a function void DisplayMatrix(char** matrix, const int & rows, const int &cols) that displays the matrix in proper format.

iv) Write a function that does the following: Creates three dynamic char arrays namely alphabets, numbers, and specialchar.(Define the sizes yourself). Iterate the 2D array and separate alphabet elements and save them in the alphabets array, separate number elements and save them in numbers array and separate special character elements and save them in the specialchar array. The function returns the three arrays alphabets, numbers, and specialchar. Note: The three arrays should not consume any extra space. Resize the arrays accordingly. For example, if the Sample Matrix is

A 1 v @

+ 9 s 6

P # ^ 4

Your function will return the following arrays:

alphabets = A v s P

numbers = 1 9 6 4

specialchar = @ + # ^